

The Auth Plugin

Authentication for the AlfacodeTeam PhpServicePlatform

Sessions • JWT • Personal Access Tokens • Refresh Tokens
Guards • Providers • Remember-me • Password Reset

A complete guide to using and implementing `Plugins\Auth`

solves: `auth.identity`

GDA-native • verification in the SecurityGateway • issuance in the service layer

Contents

1	What the Auth plugin is	2
1.1	The five authentication methods	2
2	Directory map	2
3	The principal: Identity	3
4	Verification: the SecurityGateway layers	3
5	Issuance: AuthServiceContract	4
6	Guard — the read-only projection	5
6.1	Hierarchical scope inheritance	5
7	AuthManager — named guards + providers	5
7.1	Pieces	6
7.2	Driver “scan”	6
7.3	StatefulSessionGuard	6
8	Session login (web / AJAX)	6
8.1	Remember-me (recaller cookie)	7
9	Personal access tokens (self-service)	7
10	Refresh tokens (revocable sessions)	7
11	Transient token (first-party SPA)	8
12	Password reset	8
13	Exceptions	8
14	Controller ergonomics	9
15	HTTP route reference	9
16	Configuration	9
16.1	Environment (module.json config[])	9
16.2	Guards + providers (config/auth.php)	10
17	Wiring checklist	10
18	End-to-end recipes	10
18.1	Protect a route for any authenticated caller	10
18.2	Web login with remember-me	10
18.3	Mobile / SPA with refresh tokens	10
18.4	First-party SPA transient token	10
18.5	Programmatic API access (personal access token)	11
19	Testing	11
20	Rules — do and don’t	11

1 What the Auth plugin is

`Plugins\Auth` (solves: `auth.identity`) is the **single home for authentication** on the platform: it decides *who* a caller is and hands you ergonomic ways to work with that identity. It does **not** do authorization policy (that belongs in your service layer) and it is **not** the multi-tenant control plane (that is `Plugins\Tenancy`).

The plugin is built on one clean split that you must keep in mind everywhere:

- **Issuance** — minting credentials (JWTs, personal access tokens, sessions, refresh tokens) lives in the *service layer* (`AuthServiceContract`, `RefreshTokenServiceContract`).
- **Verification** — checking an incoming credential lives in the *SecurityGateway* as `SecurityLayer` classes that run *before any module loads*. A denied request costs zero module work.

The security principal produced by verification is the kernel's `Identity` value object — immutable, and carried on the request. Every higher-level convenience in this plugin (`Guard`, `AuthManager`, `AuthUserProxy`) is a projection over that `Identity`; none of them replace it.

1.1 The five authentication methods

Method	Credential	Verified / issued by
JWT (Bearer)	Authorization: Bearer <jwt>	<code>JwtAuthLayer</code> verifies; <code>AuthService::issueJwt</code> mints
Personal access token	Authorization: Bearer <id>.<secret>	<code>PersonalAccessTokenLayer</code> verifies; <code>AuthService::createPersonalAccessToken</code> mints
Session (web/AJAX)	session cookie + <code>remember_web</code> cookie	<code>SessionAuthStage</code> rebuilds <code>Identity</code> ; <code>AuthService::startSession</code> opens it
Refresh token	opaque token in POST body	<code>RefreshTokenService::rotate</code> verifies+rotates; <code>issue</code> mints
Transient token	session → short JWT	<code>TransientTokenController</code> mints from a live session

2 Directory map

<code>plugins/Auth/</code>	
<code>module.json</code>	single source of truth (routes, config, exposes)
<code>config/auth.php</code>	named guards + providers (<code>AuthManager</code>)
<code>Support/</code>	
<code>helpers.php</code>	<code>auth_config()</code>
<code>Token.php</code>	random / hash / ulid helpers
<code>API/</code>	
<code>Guard.php</code>	read-only <code>Identity</code> projection
<code>ScopeInheritance.php</code>	hierarchical colon-scope matching
<code>Contracts/</code>	
<code>AuthServiceContract.php</code>	issuance + <code>guard()/tokensFor()</code>
<code>RefreshTokenServiceContract.php</code>	issue/rotate/revoke sessions
<code>DTOs/</code>	
<code>TokenDTO.php</code>	a PAT (no secret)
<code>PersonalAccessTokenResult.php</code>	one-time mint result
<code>RefreshTokenIssued.php / RefreshRotation.php</code>	
<code>Domain/</code>	
<code>Entities/PersonalAccessToken.php, RefreshTokenRecord.php</code>	
<code>ValueObjects/Recaller.php</code>	remember-me cookie value

Exceptions/	Authentication/Authorization/MissingScope/ InvalidAuthToken/InvalidRefreshToken
Application/	
Services/AuthService.php, RefreshTokenService.php	
Auth/AuthManager.php, ModelUserProvider.php, AuthUserProxy.php,	
GuardAccessor.php, GuardBehaviour.php, StatefulSessionGuard.php,	
PersonalAccessTokenFactory.php, PasswordResetBroker.php	
Ports/UserProvider.php, GuardDriver.php, GuardContext.php,	
StatefulGuard.php, SupportsBasicAuth.php, RefreshTokenStore.php,	
PasswordBroker.php, Authenticatable.php	
Infrastructure/	
Http/Controllers/SessionAuthController, PersonalAccessTokenController,	
TransientTokenController, AuthTokenController	
Http/Stages/SessionAuthStage.php	
Auth/Drivers/SessionDriver, JwtDriver, TokenDriver, RequestDriver	
Persistence/PersonalAccessTokenRepository, RefreshTokenRepository	
Cli/PruneAccessTokensCommand	
Security/	
JwtAuthLayer.php	wired into withSecurity([...])
PersonalAccessTokenLayer.php	
database/migrations/	personal_access_tokens, refresh_tokens

3 The principal: Identity

Everything keys on the kernel value object Identity (immutable):

```
final readonly class Identity {
    public string $userId;           // '' for a guest
    public string $tenantId;         // '' = central / unscoped
    public array $roles;             // list<string>
    public array $permissions;       // list<string> (PAT abilities or OAuth scopes)
    public string $tokenType;        // 'jwt' | 'api_key' | 'session' | 'none'

    public function hasRole(string $r): bool;
    public function hasPermission(string $p): bool; // honours '*'
    public function isGuest(): bool;
}
```

The SecurityGateway attaches it to the request; you read it with `$request->identity()`. Do **not** invent another principal type — the whole plugin projects over this one.

4 Verification: the SecurityGateway layers

Two layers ship with the plugin. A project wires them into the kernel builder; they run in order, before any module loads, and **never throw** — they return a SecurityVerdict.

```
// app/bootstrap/base.php
use Plugins\Auth\Security\JwtAuthLayer;
use Plugins\Auth\Security\PersonalAccessTokenLayer;

return Kernel::configure()
    ->withSecurity([
        new FirewallLayer(...),
        new RateLimiterLayer(...),
        new CsrfTokenLayer(...),
        new JwtAuthLayer(                               // Bearer <jwt>
            secret: env('JWT_SECRET'),
        )
    ])
```

```

        algo:      env('JWT_ALGO', 'HS256'),
        issuer:    env('JWT_ISSUER'),
        audience: env('JWT_AUDIENCE'),
        revocations: $cachePortInstance, // jti deny-list
    ),
    new PersonalAccessTokenLayer($databasePortInstance), // Bearer <id>.<secret>
]);

```

- **JwtAuthLayer** — verifies signature, iss/aud, expiry (with leeway), and consults a jti deny-list via CachePort so a token can be revoked before it expires. Pin a *single* algorithm; never let the token's alg choose the verifier.
- **PersonalAccessTokenLayer** — hashes the presented <id>.<secret>, looks it up in `personal_access_token`, enforces `expires_at`, and loads the token's abilities as `Identity.permissions`.
- No header → allow as guest. Bad credential → `deny(401)`.

Session auth cannot be a SecurityLayer — the session is only opened at `after.load`, which is after the gateway. So the plugin registers `SessionAuthStage` at `after.load` priority 22 (after `StartSessionStage`, before the route auth filter). It attaches a `tokenType: 'session'` Identity when a logged-in session exists, so the same auth route filter protects token *and* session callers uniformly. A request that already carries a token Identity is left untouched (token wins).

5 Issuance: AuthServiceContract

The published contract. Issue credentials here; never verify here.

Method	Notes
<code>issueJwt(userId, claims, ttl): string</code>	adds iat/nbf/exp/jti, plus iss/aud when configured; asymmetric algos sign with the private key
<code>revokeJwt(jti, ttl): void</code>	deny-lists a jti via CachePort (key <code>auth:jwt:revoked:<jti></code>)
<code>createPersonalAccessToken(userId, name, abilities, ttl)</code>	returns {id, token}; plaintext ONCE; only the hash is stored
<code>revokePersonalAccessToken(id): void</code>	—
<code>tokensFor(userId): list<TokenDTO></code>	a user's PATs, no secrets
<code>guard(Request): Guard</code>	read-only projection over the request Identity
<code>startSession(session, userId, roles, perms, tenantId)</code>	rotates the session id (fixation defence), stores the identity
<code>endSession(session): void</code>	invalidate + rotate
<code>hashPassword / verifyPassword</code>	bcrypt/argon2 via HashingPort, timing-safe

```

// Mint a JWT for a verified user
$jwt = $auth->issueJwt('user-123', [
    'roles'      => ['admin'],
    'permissions' => ['invoice:create'],
    'tnt'        => 'tenant-9', // scope hint (routing), NOT authorization
], ttlSeconds: 3600);

// Revoke it early (before exp) by deny-listing its jti
$auth->revokeJwt($jtiFromToken, ttlSeconds: 3600);

```

6 Guard — the read-only projection

Plugins\Auth\API\Guard is the ergonomic, stateless replacement for the old Laravel-style Auth facade. It reads the resolved Identity off the request; it holds no mutable state and no globals.

```
$guard = Guard::fromRequest($request); // or $auth->guard($request)

$guard->check(); // authenticated?
$guard->guest();
$guard->id(); // user id, or ''
$guard->tenantId();
$guard->via(); // 'jwt' | 'api_key' | 'session' | 'none'
$guard->viaToken(); // Bearer credential (jwt or api_key)?
$guard->viaSession();
$guard->hasRole('admin');
$guard->hasPermission('invoice:create');
$guard->hasScope('reports:export'); // hierarchical -- see next section

// Test/setup helper
$g = Guard::actingAs('u1', ['reports'], roles: ['analyst']);
```

6.1 Hierarchical scope inheritance

Scopes and PAT abilities are **colon-hierarchical**: a held scope satisfies every descendant. The logic lives in ScopeInheritance::satisfies and powers Guard::hasScope, AuthUserProxy::tokenCan and TokenDTO::can.

```
ScopeInheritance::satisfies(['admin'], 'admin:users:write'); // true (ancestor)
ScopeInheritance::satisfies(['admin:users'], 'admin:users:write'); // true
ScopeInheritance::satisfies(['reports'], 'billing'); // false
ScopeInheritance::satisfies(['adm'], 'admin'); // false (not a
// colon boundary)
// '*' grants everything; 'scope:'-namespaced (OAuth2) and bare (PAT) both match.
```

7 AuthManager — named guards + providers

For multi-guard apps (web session + token API + jwt), AuthManager manages named **guards** and user **providers** from config/auth.php. It is the GDA-native rework of the old AuthManager: no global auth. alias, no kernel()/config() reach-ins, and the kernel Identity remains the principal (guards resolve an AuthUserProxy that emits an Identity).

```
// config/auth.php
return [
    'defaults' => ['guard' => 'web', 'provider' => 'users'],
    'guards' => [
        'web' => ['driver' => 'session', 'provider' => 'users'],
        'api' => ['driver' => 'token', 'provider' => 'users'],
        'jwt' => ['driver' => 'jwt', 'provider' => 'users'],
        'request' => ['driver' => 'request', 'provider' => 'users'],
    ],
    'providers' => ['users' => ['driver' => 'model']],
];
```

```
$manager->guard(); // default guard accessor
$manager->guard('api')->user(); // ?Authenticatable (an AuthUserProxy)
$manager->guard('jwt')->identity(); // kernel Identity
```

```

$manager->user(); // default guard's user
$manager->provider('users'); // a named UserProvider

// custom guards / providers (runtime, request-scoped)
$manager->extend('sso', fn($req, $name, $cfg) => new GuardAccessor(...));
$manager->extendProvider('ldap', fn($name) => new LdapUserProvider(...));
$manager->forgetGuards(); // Swoole: clear per-request cache
$manager->issueToken('u1', ['roles' => ['user']], 3600); // -> JWT via AuthService

```

7.1 Pieces

- **UserProvider / ModelUserProvider** — resolves users from the central identity store (**UserServiceContract**), *not* an ORM. **retrieveByCredentials** does the full timing-safe verify (the store hides the password hash), so there is no separate hash-leaking step.
- **AuthUserProxy** — a lightweight current-user object carrying id/username/email + security context. It emits a kernel **Identity** via **identity()** and exposes the **HasApiTokens** surface (**tokens()/token()/tokenCan()/createToken()**). It is *not* the principal.
- **GuardDriver** implementations under **Infrastructure/Auth/Drivers** — **session** (session store), **jwt/token** (rehydrate the gateway verdict by **tokenType**), **request** (credential-agnostic).

7.2 Driver “scan”

AuthManager::drivers() filesystem-scans **Infrastructure/Auth/Drivers/*.php** for **GuardDriver** implementations, keyed by **driverName()**, *once per process, cached*. This is a deliberate, documented exception to the GDA “nothing auto-discovered at runtime” rule — it runs at boot, never on the hot path.

7.3 StatefulSessionGuard

The interactive login/logout guard (**StatefulGuard** + **SupportsBasicAuth**):

```

$guard->attempt(['email' => $e, 'password' => $p], remember: true);
$guard->validate($credentials); // check, no session
$guard->once($credentials); // authenticate for one request
$guard->loginUsingId('u1', remember: true);
$guard->login($authenticatable, remember: true);
$guard->logout(); // clears session + remember cookie + token
$guard->logoutOtherDevices($password); // re-verify, then rotate remember token
$guard->viaRemember(); // was this resolved from a recaller cookie?
$guard->basic('email'); // HTTP Basic -> null on success, 401 Response
on fail
$guard->onceBasic('email');

```

8 Session login (web / AJAX)

Endpoints from **SessionAuthController**:

```

POST /auth/login    { identifier, password, remember? } -> 200 {user} | 401
POST /auth/logout                                     -> 204
GET  /auth/me                                           -> 200 identity | 401

```

Login verifies the credential through the User module (timing-safe + logout), then **startSession** opens a session (rotating the id). CSRF is the kernel's **CsrfTokenLayer** (these routes are outside **/api**).

8.1 Remember-me (recaller cookie)

POST /auth/login with `remember=true` issues an encrypted `remember_web` cookie holding a `userId|token` **recaller** (Plugins\Auth\Domain\ValueObjects\Recaller — a flat pipe string, NEVER unserialized). When a later request has no live session, `SessionAuthStage`:

1. reads + decrypts the cookie (via the Cookie plugin);
2. resolves the user by the token's SHA-256 hash (`UserServiceContract::findByRememberToken`), rejecting a mismatched owner id or a stale token;
3. re-opens the session (rotating the id) and attaches a session Identity;
4. **rotates** the token + cookie, so a stolen cookie is a single-use window.

Logout clears the stored token and expires the cookie. The `remember_token` column already exists on the central `users` table.

9 Personal access tokens (self-service)

First-party user API keys (GitHub-PAT style, Bearer <id>.<secret>), owner-scoped to the caller's Identity. These are *not* OAuth clients and are *not* used by session login.

```
GET    /auth/tokens      -> list my tokens (no secrets)
POST   /auth/tokens      { name, abilities[], ttl? } -> 201 { id, token } (once)
DELETE /auth/tokens/{id} -> 204 (only my token, else 404)
```

```
// Programmatic minting
$result = $auth->createPersonalAccessToken('u1', 'ci-deploy', ['deploy:run'], 86400);
// $result = ['id' => '...', 'token' => '<id>.<secret>'] -- show once

// From the current user object (HasApiTokens on AuthUserProxy)
$user    = $manager->user();
$tokens  = $user->tokens();           // list<TokenDTO>
$user->tokenCan('deploy:run');        // hierarchical
$user->createToken('backup', ['storage:read']);
```

Only the SHA-256 is persisted; expiry is enforced in PHP; abilities become `Identity.permissions` at verify time. Prune expired tokens with the CLI command `auth:tokens:prune`.

10 Refresh tokens (revocable sessions)

Relocated from Tenancy — authentication is not tenancy. `RefreshTokenServiceContract` provides revocable long-lived first-party sessions paired with a short access JWT.

```
$issued = $refresh->issue('u1', tenantId: null, device: $ua, ip: $ip);
// $issued->token -- raw refresh token, shown ONCE (only the hash is stored)

$rot = $refresh->rotate($rawToken, $ip);
// -> RefreshRotation { accessToken, expiresIn, refreshToken, refreshExpiresAt,
    tenantId }

$refresh->revoke($rawToken);           // logout this session
$refresh->revokeAllForUser('u1');      // logout everywhere
```

One-time-use rotation with reuse detection. Each `issue()` roots a token *family* (`family_id`); every rotation stays in the family and atomically revokes the presented token. Replaying an already-revoked token, or losing a rotation race, is treated as **reuse**: the whole family is burned and a 401

is returned. Only the SHA-256 of each token is stored.

Tenant-agnostic. `tenantId` rides through as a passthrough hint for the access token's `tnt` claim, but is **never re-verified** on refresh. The tenant-seat check (a revoked seat cannot get back in) lives in the Tenancy selection flow, not here.

```
POST /auth/refresh      { token } -> new access JWT + rotated refresh token | 401
POST /auth/refresh/logout { token } -> 204
```

11 Transient token (first-party SPA)

POST `/auth/token/refresh` (auth-filtered). A session-authenticated SPA mints a short-lived (900s) JWT carrying the session identity's real roles/permissions. It is the modern, *scoped* replacement for Passport's blanket "all scopes pass" transient token — a Bearer/PAT caller (non-session) is refused.

```
POST /auth/token/refresh -> { access_token, token_type: "Bearer", expires_in: 900 }
```

12 Password reset

`PasswordBroker` (implementation `PasswordResetBroker`) is `CachePort`-backed and enumeration-safe. No token table — the cache is the store.

```
$res = $broker->sendResetLink('alice@example.com');
// -> ['status' => PasswordBroker::RESET_LINK_SENT, 'token' => '...', 'userId' => '...']
// (email the token; unknown user -> INVALID_USER with no token; rapid repeat -> THROTTLED)

$broker->validateToken('alice@example.com', $token);
$status = $broker->reset('alice@example.com', $token, 'N3wPassw0rd!');
// PASSWORD_RESET on success -- sets the password via UserServiceContract::
//   resetPassword
// (which also clears remember tokens) and burns the one-time token.
```

Statuses: `RESET_LINK_SENT`, `PASSWORD_RESET`, `INVALID_USER`, `INVALID_TOKEN`, `THROTTLED`. Tokens are hashed, one-time-use, and compared in constant time.

13 Exceptions

Exception	HTTP	When
<code>AuthenticationException</code>	401	no / invalid credential (carries the guards tried)
<code>AuthorizationException</code>	403	authenticated but denied; <code>asNotFound()</code> masks as 404
<code>MissingScopeException</code>	403	valid token lacks a required scope (<code>scopes()</code>)
<code>InvalidAuthTokenException</code>	401	<code>::different()/expired()/revoked()</code>
<code>InvalidRefreshTokenException</code>	401	<code>::invalid()/reuseDetected()</code>

Security layers **never throw** — they return a `SecurityVerdict`. These exceptions are for the service/controller layers.

14 Controller ergonomics

Two project-layer concerns (both build on `HasRequest`; the base `ApiController/ViewController` already implement `RequestAware`):

- `InteractsWithAuth` — lightweight Identity projection: `$this->guard(), $this->identity(), $this->authId(), $this->tokenCan('write')`. Works even without the Auth module loaded.
- `InteractsWithAuthManager` — named guards + rich user: `$this->auth('api')->user(), $this->authUser()`. The route must declare `"requires": ["auth.identity"]`.

```
final class ReportController extends ApiController
{
    use InteractsWithAuth;

    public function export(): Response
    {
        if (!$this->tokenCan('reports:export')) { // hierarchical
            return Response::forbidden();
        }
        return Response::json(['ok' => true]);
    }
}
```

15 HTTP route reference

Method	Path	Purpose (filters)
POST	/auth/login	session login (<code>throttle:10,1</code>)
POST	/auth/logout	session logout
GET	/auth/me	current identity
GET	/auth/tokens	list my PATs (<code>auth</code>)
POST	/auth/tokens	mint a PAT (<code>auth, throttle:20,1</code>)
DELETE	/auth/tokens/{id}	revoke my PAT (<code>auth</code>)
POST	/auth/token/refresh	transient SPA token (<code>auth</code>)
POST	/auth/refresh	refresh-token rotation (<code>throttle:30,1</code>)
POST	/auth/refresh/logout	revoke a refresh session

16 Configuration

16.1 Environment (`module.json config[]`)

Key	Meaning
JWT_SECRET	HMAC secret (HS* algorithms)
JWT_ALGO	default HS256; RS/ES/PS for asymmetric
JWT_ISSUER / JWT_AUDIENCE	bound + verified claims
JWT_PRIVATE_KEY / _FILE	PEM signing key (file form keeps keys off the env)
JWT_KID	key id for rotation
AUTH_PAT_TABLE	PAT table override (default <code>personal_access_tokens</code>)
AUTH_REFRESH_TTL	refresh-token lifetime (default 30d)
AUTH_REFRESH_ACCESS_TTL	paired access-JWT lifetime (default 900s)

Always read config through the `env()` helper — never `getenv()` for a `.env` value in first-party code.

16.2 Guards + providers (config/auth.php)

The guard/provider maps live in `config/auth.php`, read via `auth_config()` (a project copy at `projects/<name>/config/auth.php` wins over the plugin default). See the AuthManager section for the shape.

17 Wiring checklist

1. Add `\Plugins\Auth\Provider::class` to the project bootstrap `->withModules([...])`.
2. Wire the verification layers in `->withSecurity([...])` (`JwtAuthLayer`, `PersonalAccessTokenLayer`) — these are not auto-registered because they need port instances.
3. Ensure the dependencies are available: `database.management`, `crypto.services` (Hashing-Port), `user.management`. Sessions need the essential Session + Cookie plugins; `revokeJwt/refresh`-revocation need `CachePort`.
4. Run migrations: `personal_access_tokens` and `refresh_tokens` (central connection).
5. A route that uses the AuthManager/PAT surface declares `"requires": ["auth.identity"]`; protect routes with the `auth` filter.

18 End-to-end recipes

18.1 Protect a route for any authenticated caller

Declare the `auth` filter on the route; it is satisfied by a JWT, a PAT, or a logged-in session (thanks to `SessionAuthStage`). Read the caller with `$this->guard()` or `$this->identity()`.

18.2 Web login with remember-me

```
// POST /auth/login { identifier, password, remember: true }
// SessionAuthController::login -> UserService::verifyCredentials -> AuthService::
    startSession
// + queues the encrypted remember_web recaller cookie.
// Later requests with no session are resurrected + rotated by SessionAuthStage.
```

18.3 Mobile / SPA with refresh tokens

```
// 1. After verifying credentials, issue a refresh token + first access JWT:
$issued = $refresh->issue($userId, device: $ua, ip: $ip); // store $issued->token
    client-side
$access = $auth->issueJwt($userId, ['roles' => $roles], 900);

// 2. When the access JWT expires, rotate:
// POST /auth/refresh { token: <refresh> } -> { accessToken, refreshToken, ... }
// Replace BOTH tokens client-side. A replayed old refresh token 401s the family.
```

18.4 First-party SPA transient token

```
// Browser is session-authenticated (cookie). To call Bearer APIs from JS:
```

```
// POST /auth/token/refresh -> { access_token, expires_in: 900 }
// Send it as Authorization: Bearer <access_token>; refresh it while the session lives.
```

18.5 Programmatic API access (personal access token)

```
// User mints a token in a dashboard:
// POST /auth/tokens { name: "ci", abilities: ["deploy:run"] } -> { token } (once)
// CI sends: Authorization: Bearer <id>.<secret>
// PersonalAccessTokenLayer verifies it and loads abilities as permissions.
```

19 Testing

Unit-test the service/guards with in-memory fakes — never real infrastructure. The plugin ships a full suite (Guard, ScopeInheritance, AuthService, StatefulSessionGuard, AuthManager, PasswordResetBroker, refresh tokens, the management controllers). Key fakes: `FakeAuthService`, `FakeUserService`, `FakeSession`, `RecordingDatabasePort`.

```
$guard = Guard::actingAs('u1', ['admin']);
self::assertTrue($guard->hasScope('admin:users:write')); // inheritance

$svc = new RefreshTokenService($inMemoryStore, new FakeAuthService(), accessTtl: 900);
$issued = $svc->issue('u1');
$rot     = $svc->rotate($issued->token);
$this->expectException(InvalidRefreshTokenException::class);
$svc->rotate($issued->token); // reuse -> family burned
```

20 Rules — do and don't

Do

- Verify in `SecurityLayers` (the gateway); issue in `AuthService`. Never mix the two.
- Pin a single JWT algorithm; use asymmetric (RS/ES/PS) when verifiers must not hold the signing secret.
- PATs: store only the hash, return plaintext once, enforce `expires_at`.
- Session login *after* credential verification; rotate the session id.
- Remember-me / refresh: store only the hash, rotate on every use, one-time-use with family reuse detection.
- Treat scopes as hierarchical (ancestor $\supseteq$ descendant); use `hasScope`, never bare string equality.

Don't

- A `SecurityLayer` that throws — always return a `SecurityVerdict`.
- Trust a `tnt` claim as authorization — it is a routing hint; authz keys on (userId, tenantId, role/permission).
- Re-check tenant-seat membership on refresh — refresh is tenant-agnostic; the seat check is at tenant-select.
- Unserialize a recaller/cookie value — the recaller is a flat `id|token` string.
- Confuse `personal_access_tokens` (user API keys) with `oauth_clients` (applications) — different things.

- `getenv()` for a `JWT_*` / `AUTH_*` value — use `env()`.

Reference guide: `docs/ai-context/25_AUTH.md` • OAuth server: `Plugins\OAuth2`
(`docs/ai-context/26_OAUTH2.md`)